

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Name: B. Tharun Kumar Reddy

Reg no: 192565015

Course code: CSA5803

Course name: DEVSECOPSENGINEERINGAND APPLICATIONSECURITY

Simulate Static Application Security Testing (SAST) Using Synthetic Data

Introduction

Static Application Security Testing (SAST)

- SAST is a security testing technique used to find vulnerabilities in source code.
- It analyzes the application **without executing the program**.
- SAST can detect security problems during the development stage.
- In this project, we simulate SAST using **synthetic data**.
- An **e-commerce website** is used as the real-time scenario.

E-Commerce Application

The application allows customers to:

- Create an account
 - Login
 - Search for products
 - Add products to cart
 - Make online payments
 - View order details
-

Real-Time E-Commerce Scenario

Scenario

Consider an online shopping website called **ShopEasy**.

A developer creates a login system where users enter their username and password.

Example of vulnerable source code:

```
username = input("Enter username: ")
```

```
password = "admin123"
```

```
query = "SELECT * FROM users WHERE username='" + username + "'"
```

Security Problems

The source code contains:

1. **Hard-coded password**
2. **Possible SQL Injection**
3. **No proper input validation**

If this code is deployed without testing, attackers may exploit these weaknesses.

Synthetic Data Used for Testing

What is Synthetic Data?

Synthetic data is **artificially created data** used for testing purposes.

For this simulation, we do not use real customer information.

Example Synthetic Data

Username	Password	Product
tharun01	Test@123	Laptop
user02	Demo@456	Mobile
customer03	Sample@789	Headphones

Synthetic Vulnerable Code

```
username = "tharun01"
```

```
password = "Test@123"
```

query = "SELECT * FROM users WHERE username='" + username + "'"

This data is created only for demonstrating the SAST process.

SAST Simulation Process

How the SAST Simulation Works

1. Source Code Creation

Developer creates the e-commerce application code.

↓

2. SAST Scanning

The source code is scanned for suspicious patterns.

↓

3. Vulnerability Detection

The scanner identifies insecure coding practices.

↓

4. Vulnerability Classification

Issues are classified as High, Medium, or Low severity.

↓

5. Security Report

A report is generated for the developer.

Example

SAST SCAN RESULT

SQL Injection → HIGH
Hard-coded Password → HIGH
Input Validation → MEDIUM

SAST Results and Fixes

Vulnerability Report

Vulnerability	Severity	Recommended Fix
SQL Injection	High	Use parameterized queries
Hard-coded Password	High	Use secure password storage
Missing Input Validation	Medium	Validate user input
Sensitive Data Exposure	Medium	Protect sensitive information

Secure Code Example

Instead of directly joining user input with SQL:

```
query = "SELECT * FROM users WHERE username=?"
```

The application should use **parameterized queries** and proper security controls.

Result

After fixing the vulnerabilities:

Scan Again → Fewer/No Critical Issues → Safer Application

Conclusion and Future Scope

Conclusion

- SAST helps detect security vulnerabilities before an application is deployed.
- The e-commerce scenario demonstrates how vulnerable code can be identified.
- Synthetic data provides a safe way to perform security testing.
- SQL Injection and hard-coded passwords were identified as major issues.
- Developers can fix vulnerabilities before releasing the application.

Future Scope

1. Use real SAST tools such as SonarQube or Semgrep.

2. Add more vulnerability categories.
3. Automate security scanning.
4. Integrate SAST into CI/CD pipelines.
5. Generate automatic security reports.
6. Perform continuous security testing during development.

Final Flow

**E-Commerce Code → Synthetic Data → SAST Scan → Vulnerability Detection → Fix
→ Re-Scan → Secure Application**